

SMC CLINT — Functional Specification

SMC CLINT — Functional Specification

Document: 01_CLINT_Specification.md **Module:** smc::clint (SystemC/TLM-2.0 Loosely-Timed model, CCI-parameterised) **Spec base:** RISC-V Privileged Architecture — mtime / mtimecmp / msip **Status:** Frozen for SMC bring-up; tracks clint.rdl and riscv_clint0.c **Companion docs:** - 02_CLINT_LowLevel_Design.md — internal SystemC implementation - 03_CLINT_Test_Plan.md — verification strategy & test list

Contents

1. Purpose & scope
 2. Conformance & references
 3. Feature summary
 4. Configuration parameters
 5. Block diagram & port list
 6. Theory of operation
 - 6.11 Real-world applications
 7. Register map
 8. Functional behaviour
 9. Reset behaviour
 10. Bus interface (TLM-2.0 / AXI4-Lite)
 11. Error handling
 12. Programming model
 13. Compliance matrix
 14. Revision history
 15. Glossary
-

1. Purpose & scope

The SMC CLINT (Core-Local Interruptor) provides per-hart **machine timer interrupts** and **machine software interrupts** to the four Rocket cores in the SMC sub-system. It is **the only path** from software to the mip.MTIP and mip.MSIP CSR bits — the PLIC handles every other interrupt source.

This specification defines the **externally-observable behaviour** of the SMC CLINT: configuration, register map, interrupt-flow semantics, reset, and bus contract. It is the contract that:

- Firmware (fw/smc/common/drivers/riscv_clint0.c) programs against,
- The RTL (0CAH{1,4}COREcluster_CLINT.sv, generated from Chipyard / Rocket-Chip) implements at gate level, and
- The SystemC model (peripherals/clint/) implements at transaction level.

Internal implementation choices (SC processes, MTIME tick scheduling, single-driver scheme, CCI parameter wiring) are documented in [02_CLINT_LowLevel_Design.md](#).

2. Conformance & references

Source	Authority
RISC-V Privileged Architecture v1.12 §3.2.1 (<code>mtime</code> , <code>mtimecmp</code>)	Protocol semantics
RISC-V Privileged Architecture v1.12 §3.1.9 (<code>mip.MSIP</code> , <code>mip.MTIP</code>)	Output mapping
<code>hw/smc/smc_cpu/data/registers/rdl/clint.rdl</code>	Ground-truth register map
<code>hw/smc/smc_cpu/chipyard_generated_files/{1,4}core/OCAH{1,4}CORECluster_CLINT.sv</code>	Functional reference RTL
<code>fw/smc/common/drivers/riscv_clint0.c</code> , <code>riscv_clint0.h</code>	Firmware driver (must run unmodified)
<code>tt-oca-hw.pdf</code> §6.6.6 “RISC-V CLINT and Precise System Management Timing”	SMC-specific role
<code>tt-oca-hw.pdf</code> §6.6.3 (Table 17)	Address window <code>0xC800_0000</code> – <code>0xC800_FFFF</code>
<code>01_PLIC_Specification.md</code>	Sister IP — same SMC modelling conventions

The model is **bit-compatible** with the RDL register layout and protocol- compatible with the RISC-V Privileged Architecture. All deviations from the Chipyard RTL are listed explicitly in §13.

3. Feature summary

Feature	Value / behaviour
Standard	RISC-V Privileged Arch. (machine-mode timer + SW interrupt)
Harts	4 (default; configurable 1..4095 via CCI)
Per-hart MSIP	1 bit (bit[0]); bits[31:1] reserved (RAZ/WI)
Per-hart MTIMECMP	64-bit unsigned
Global MTIME	64-bit unsigned, monotonically increasing
MTIME tick	Configurable period (default 100 ns $\Rightarrow$ 10 MHz); 0 disables auto-tick
MTIP[h]	$\text{MTIME} \geq \text{MTIMECMP}[h]$ (level, unsigned compare)
MSIP[h]	$\text{MSIP}[h].\text{bit}[0]$ (level)
Bus interface	TLM-2.0 LT target socket (AXI4-Lite, 32 / 64-bit)
Window size	64 KiB (0×10000)
DMI	Not granted (atomic 64-bit observation needed)
Reset polarity	Active-low, synchronous
Reset values	$\text{MTIME} = 0$, $\text{MSIP}[h] = 0$, $\text{MTIMECMP}[h] = 0 \times \text{FFFF} \dots \text{F}$
Configuration interface	SystemC CCI 1.0 ($\text{cci}::\text{cci_param}<\text{T}>$)
Per-hart isolation	MSIP, MTIMECMP, MTIP independent across harts

4. Configuration parameters

The model exposes **three real `cci::cci_param<T>` declarations**. Address-map constants are fixed by `clint.rdl` and **must not be changed** without a corresponding RDL update.

Parameter	Type	Default	Mutability	Range	Notes
<code>num_harts</code>	unsigned	4	immutable	1 .. 4095	Sizes <code>MSIP[]</code> , <code>MTIMECMP[]</code> , <code>msip_o[]</code> , <code>mtip_o[]</code> .
<code>tick_period_ns</code>	double	100.0	immutable	≥ 0.0	MTIME auto-tick period (ns). 0 disables auto-tick.
<code>access_delay_ns</code>	double	2.0	mutable	≥ 0.0	TLM <code>b_transport</code> annotated delay; can be changed at run time.

Override before construction via the CCI broker:

```
broker.set_preset_cci_value("smc.clint.num_harts",      cci::cci_value(2u));
broker.set_preset_cci_value("smc.clint.tick_period_ns", cci::cci_value(1000.0));
broker.set_preset_cci_value("smc.clint.access_delay_ns", cci::cci_value(5.0));
```

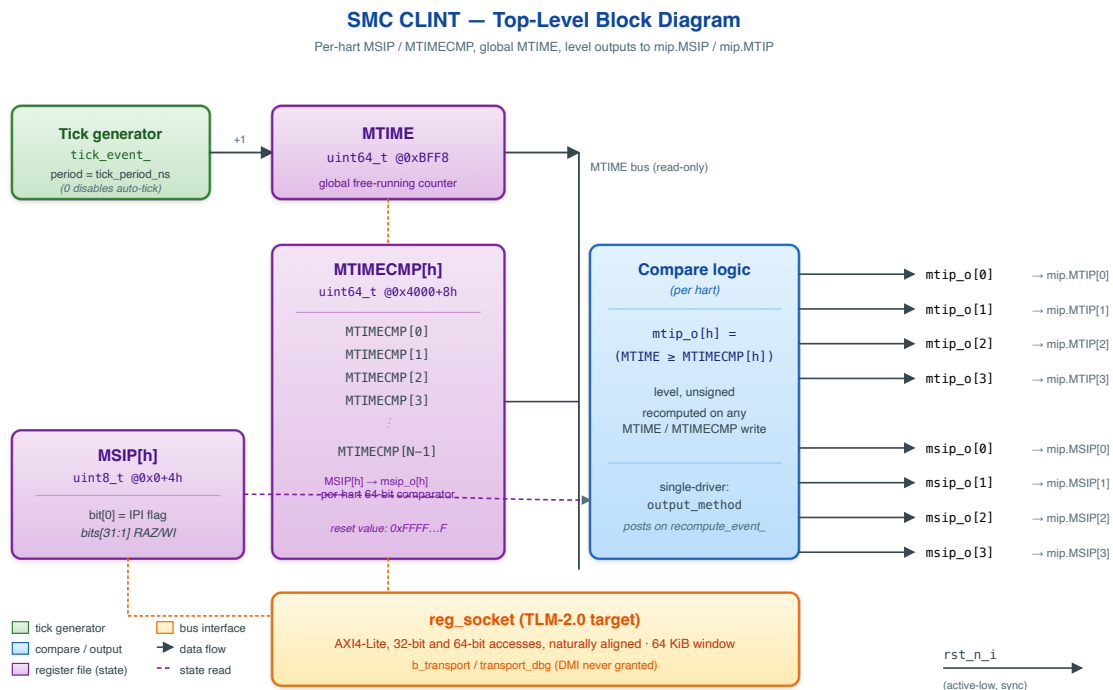
Fixed address-map constants (from `clint.rdl`):

Constant	Value	Description
MSIP_BASE	0x0000	$MSIP[h] = MSIP_BASE + 4 \cdot h$
MSIP_STRIDE	0x0004	Per-hart MSIP stride
MTIMECMP_BASE	0x4000	$MTIMECMP[h] = MTIMECMP_BASE + 8 \cdot h$
MTIMECMP_STRIDE	0x0008	Per-hart MTIMECMP stride
MTIME_OFFSET	0xBFF8	64-bit global MTIME counter
WINDOW_SIZE	0x10000	Total decoded window (64 KiB)

The SMC platform-level base address is `0xC800_0000` per `tt-oca-hw.pdf §6.6.6`; the model exposes only the byte offsets above, and the fabric is responsible for the upper-address decode.

5. Block diagram & port list

5.1 Block diagram



SMC CLINT top-level block diagram: TLM-2.0 register socket feeds the per-hart MSIP register, the per-hart MTIMECMP register, and the global 64-bit MTIME counter that increments on each `tick_period_ns` event; per-hart compare logic produces `mtip_o[h]` (level, $MTIME \geq MTIMECMP[h]$), and `MSIP[h].bit[0]` drives `msip_o[h]`; both lines bypass the PLIC and feed the cores' `mip.MSIP` / `mip.MTIP` CSRs directly.

5.2 Ports

Port	Direction	Type	Description
reg_socket	target	simple_target_socket	TLM-2.0 register-access socket (AXI4-Lite, 32 / 64-bit)
msip_o[h]	output	sc_out<bool> × N	Machine SW interrupt for hart h (active-high)
mtip_o[h]	output	sc_out<bool> × N	Machine timer interrupt for hart h (active-high)
rst_n_i	input	sc_in<bool>	Active-low synchronous reset

N = num_harts.

5.3 Hart mapping (default num_harts = 4)

h	RTL signal mapping (4-core variant)	Drives
0	auto_int_out_0_0, auto_int_out_0_1	MSIP[0], MTIP[0]
1	auto_int_out_1_0, auto_int_out_1_1	MSIP[1], MTIP[1]
2	auto_int_out_2_0, auto_int_out_2_1	MSIP[2], MTIP[2]
3	auto_int_out_3_0, auto_int_out_3_1	MSIP[3], MTIP[3]

Per `OCAH4C0RECluster_CLINT.sv`, the sub-index `_0` is MSIP and `_1` is MTIP; the model exposes them as two separate `sc_vector` ports for clarity.

6. Theory of operation

This section describes how the CLINT operates from a conceptual viewpoint, independently of the register-level details given in §7. It walks through the operating principle, the lifecycle of a single timer or software interrupt event, and the steady-state interaction between hardware and software.

6.1 Operating principle

The CLINT is a **direct, low-latency channel** from a small set of machine-mode-visible registers to the per-hart `mip.MSIP` and `mip.MTIP` CSRs. There is no arbitration, no priority, and no claim/complete handshake. Every observable output is a pure level function of the register file:

```
msip_o[h] = MSIP[h].bit[0]
mtip_o[h] = (MTIME >= MTIMECMP[h])           // unsigned 64-bit
```

The only autonomous activity is the MTIME counter, which increments by 1 every `tick_period_ns` of simulated time. Setting `tick_period_ns = 0` disables auto-tick — the test bench then drives MTIME via register writes or `dbg_set_mtime` for fully deterministic timing experiments.

The CLINT has no clock of its own from the model's standpoint. State changes occur only on:

- An MTIME tick event (auto-armed),
- A register write (MSIP / MTIMECMP / MTIME),
- A reset assertion.

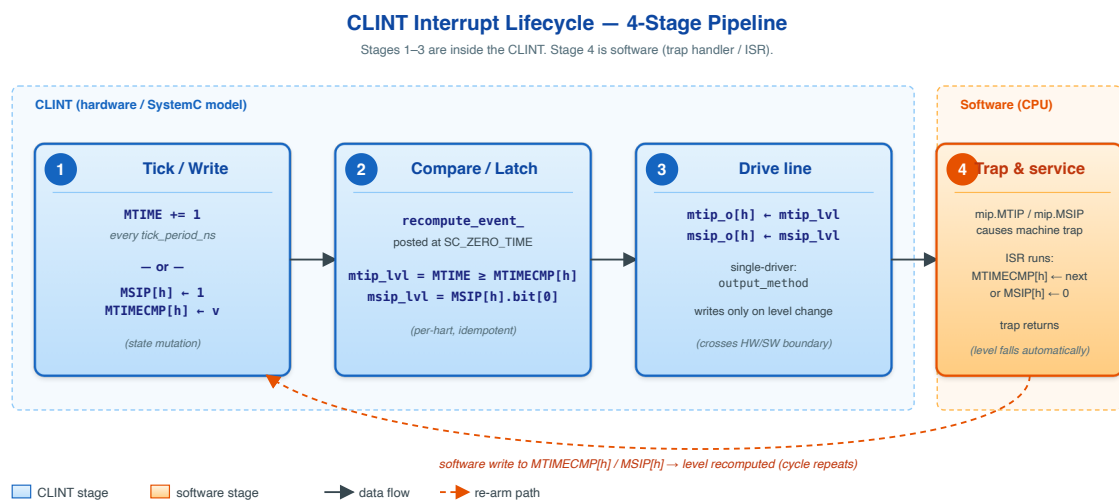
6.2 Architectural concepts

Concept	Definition
Hart	A RISC-V hardware thread; identified by a 0-based index $h \in [0, \text{num_harts})$.
MSIP[h]	32-bit register; only bit[0] is the IPI flag. Bits[31:1] are reserved (RAZ/WI).
MSIP[h] line	<code>msip_o[h] = MSIP[h].bit[0]</code> ; level-sensitive, active-high.
MTIME	64-bit free-running counter; increments by 1 every <code>tick_period_ns</code> of sim time.
MTIMECMP[h]	64-bit per-hart comparator; software-writable.
MTIP[h] line	<code>mtip_o[h] = (MTIME ≥ MTIMECMP[h])</code> ; level-sensitive, active-high.
Tick	The MTIME-increment event; period is the <code>tick_period_ns</code> CCI parameter.
Inter-processor interrupt (IPI)	A hart writing <code>MSIP[other_hart] = 1</code> to wake / signal another hart.

These seven concepts cover the entire externally-observable behaviour; everything else in this document is bookkeeping (register addressing, error responses, reset wiring) built on top of them.

6.3 Pipeline view — interrupt lifecycle

A single timer or software-interrupt event flows through the CLINT in four observable stages:

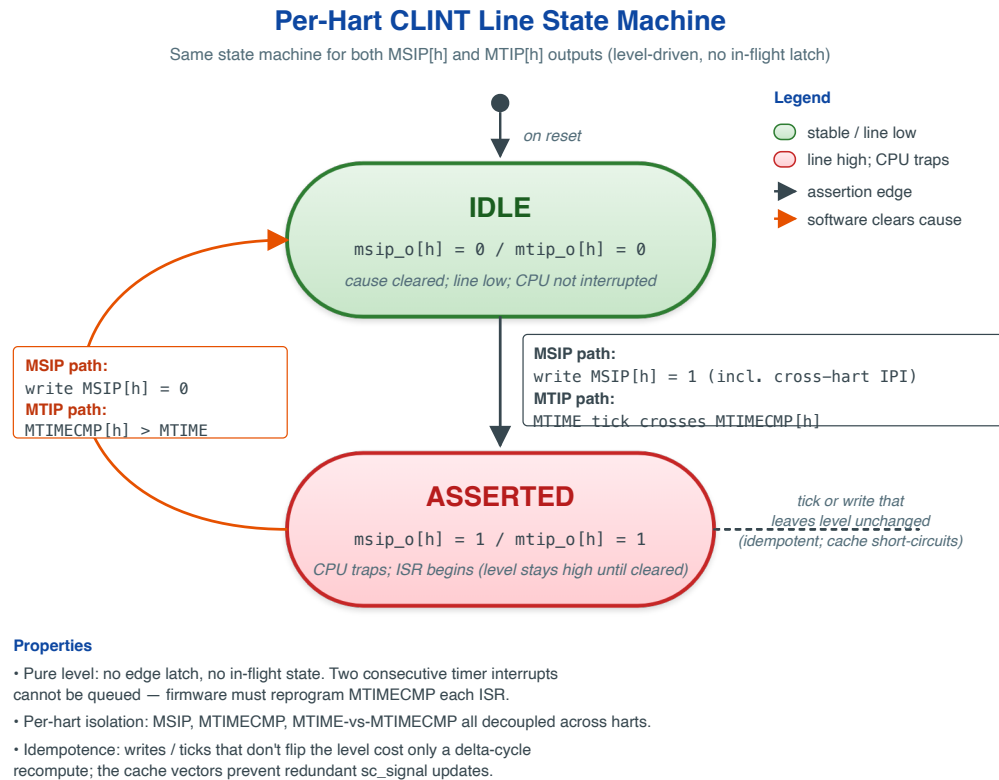


CLINT interrupt lifecycle pipeline: stages 1-3 (Tick / Write, Compare or Latch, Drive line) are inside the CLINT; stage 4 (Trap & service) happens in the core; the dashed re-arm path shows that after the ISR clears the cause ($\text{MTIMECMP} > \text{MTIME}$, or $\text{MSIP}[h] = 0$) the line returns to 0 automatically.

Stages 1-3 are entirely the CLINT's responsibility. Stage 4 happens in the core / software (the trap handler in `fw/smc/common/drivers/`); the CLINT only observes the resulting register writes that clear the cause.

6.4 Per-hart lifecycle

Each MSIP / MTIP line follows a tiny three-state pattern. Unlike the PLIC's per-source machine, the CLINT lines have **no in-flight state** because there is no claim/complete protocol — the cause is cleared purely by software writing the right register value.



Per-hart CLINT line state machine: from IDLE, an MSIP write to bit[0]=1 or an MTIME tick that crosses MTIMECMP moves the line to ASSERTED; the core takes the trap and software writes back (MSIP=0 for SW interrupt, MTIMECMP>MTIME for timer) which returns the line to IDLE; transitions are level-driven, not edge-driven.

Two key properties fall out of this state machine:

- **Level, not edge.** The line is a pure function of register state and MTIME. There is no latched event: if firmware writes `MTIMECMP[h] = MTIME` and then immediately writes `MTIMECMP[h] = MTIME + 100`, the line glitches from `0 → 1 → 0` across those two writes. Real hardware exhibits the same behaviour.
- **No queueing.** Two timer interrupts cannot be “stacked” — only the current `(MTIME, MTIMECMP[h])` relationship matters. To rate-limit timer interrupts, firmware programs `MTIMECMP[h] = MTIME + interval` inside the trap handler.

6.5 Per-hart isolation

The MSIP, MTIMECMP, and resulting MTIP / MSIP lines are **completely independent across harts**. A write to `MSIP[0]` cannot affect `MSIP[1]`, and a change in `MTIMECMP[2]` cannot affect `MTIP[3]`. MTIME is the **only** shared piece of state — by design, so all harts agree on a single time base for synchronisation.

6.6 Cross-hart software interrupts (IPI)

The intended use of MSIP is **inter-processor signalling**:

- Hart A wishing to signal hart B writes `MSIP[B] = 1`.
- This raises `msip_o[B]`, which is fed straight into hart B's `mip.MSIP` CSR.
- Hart B traps into its machine-mode software-interrupt handler, inspects shared memory to discover the message, and writes `MSIP[B] = 0` to clear the line before returning.

Because MSIP is a level-sensitive output and software clears it explicitly, multiple senders cannot lose IPIs to each other: if A and C both write `MSIP[B] = 1` before B clears it, B sees a single combined IPI and infers the rest from its own message-passing convention (a queue, a bitmask, a counter, etc.). The CLINT itself does not implement IPI multiplexing.

6.7 MTIME tick semantics

The MTIME counter advances by exactly 1 every `tick_period_ns`:

<code>tick_period_ns</code>	Effective MTIME frequency	Use case
100.0	10 MHz (default)	Realistic SMC bring-up profile.
1.0	1 GHz	Maximum-resolution timing experiments.
1000.0	1 MHz	RISC-V “RTC” convention; matches Linux defaults.
0.0	(auto-tick disabled)	Test mode; TB advances MTIME explicitly.

Software always reads the same monotonically-increasing 64-bit MTIME regardless of `tick_period_ns`. The choice of period only affects how fast simulated time advances **per unit of wall-clock simulator time**.

6.8 Hardware ↔ software interaction

The CLINT exposes exactly four classes of operation to software:

Operation	Effect
Write <code>MSIP[h]</code>	Drive <code>msip_o[h]</code> to bit[0]; bits[31:1] are dropped.
Write <code>MTIMECMP[h]</code>	Reprogram the comparator; <code>mtip_o[h]</code> recomputed.
Read <code>MTIME</code>	Snapshot the 64-bit time base. Firmware uses the rollover-safe sequence (read hi, lo, hi-again) for 32-bit hosts.
Write <code>MTIME</code>	Force-set the time base (privileged; rarely used outside debug / boot).

Every other access is RAZ/WI (in-window) or returns a TLM error response (out-of-window / misaligned / wrong-size).

Boot sequence (simplified)

1. **Reset clears MTIME and MSIP.** All `MSIP[h] = 0` and `MTIME = 0`. `mtip_o[h]` is deasserted because the model defaults `MTIMECMP[h] = 0xFFFF_FFFF_FFFF_FFFF`.
2. **Firmware programs MTIMECMP[h]** for any hart that needs timer interrupts: typically `MTIMECMP[h] = MTIME + interval`.
3. **Firmware enables external interrupts** in `mie.MTIE` / `mie.MSIE` and sets `mstatus.MIE`.
4. **Steady state.** The next MTIME crossing of `MTIMECMP[h]` raises `mtip_o[h]` and the target hart traps into its machine-mode trap handler.

Timer-interrupt ISR pattern

```
machine_timer_handler(h):
    # mip.MTIP is level: it stays asserted until we move MTIMECMP > MTIME
    next = mtimecmp[h] + interval
    if next < mtime:                # we missed deadlines
        next = mtime + interval    # skip to the next future deadline
    mtimecmp[h] = next             # writes 0xFFFFFFFF first per
                                   # __metal_driver_riscv_clint0_mtimecmp_set
                                   # to avoid spurious MTIP transients
    # implicit: mip.MTIP falls because (mtime < next)
```

The “high=0xFFFFFFFF, low, high” sequence in `__metal_driver_riscv_clint0_mtimecmp_set` is **important** on 32-bit hosts: it temporarily forces MTIMECMP very far in the future before patching the low half, preventing a transient assertion of MTIP if the old high half happened to be 0 and the new low half rolls past the current MTIME.

6.9 Edge cases

Scenario	CLINT response
MTIMECMP[h] written equal to current MTIME	mtip_o[h] asserts immediately ($\geq$ comparison).
MTIMECMP[h] = 0	mtip_o[h] asserted continuously (since $\text{MTIME} \geq 0$ always).
MTIME passes MTIMECMP between two ticks of the firmware	mtip_o[h] is observed at the next read; level, no race.
MSIP write with bits[31:1] set	Bits[31:1] silently dropped (RAZ/WI per RDL).
32-bit MTIME read straddling a 32-bit overflow	Firmware uses the hi-lo-hi sequence to guard. Model returns the current value of each half independently; firmware retries until hi is stable.
MTIMECMP write split across two 32-bit halves with the line momentarily armed	Firmware uses “hi=0xFFFFFFFF, lo, hi” sequence to avoid this. Model honours each half-word write atomically.
Reset asserted while mtip_o[h] = 1	All registers reset; mtimecmp[h] = max $\Rightarrow$ mtip_o[h] deasserts.
Read MSIP[h] immediately after writing 1	Returns 1 in the same transaction (pure register; no pipeline).

6.10 Steady-state characteristics

Quantity	Value (default config: 4 harts)
Maximum sustained tick rate	One MTIME increment per <code>tick_period_ns</code> (default 10 MHz).
Per-tick simulator overhead	One SC_METHOD invocation + N MTIP comparisons (one per hart).
Worst-case output recompute	$2 \times \text{num_harts}$ comparisons (MSIP + MTIP per hart).
MMIO access latency	<code>access_delay_ns_p_</code> (default 2 ns).
DMI	Not granted.

For a 4-hart SMC running with the default 10 MHz tick, the steady-state recompute cost is 8 comparisons $\times$ 10^7 Hz = 8×10^7 ops/s of host CPU, typically below 1 % of a single core in Release builds — small enough to ignore relative to the rest of the SMC model.

6.11 Real-world applications

The CLINT is one of the most widely exercised peripherals in any RISC-V system: every firmware that requires periodic scheduling, inter-core communication, or elapsed-time measurement touches it. The six patterns below cover the vast majority of real deployments, from bare-metal microcontrollers to multi-core Linux SoCs.

6.11.1 RTOS preemptive scheduler tick

What it is. Real-time operating systems (FreeRTOS, Zephyr, RT-Thread, RTEMS) need a periodic hardware tick to drive the preemptive scheduler. The CLINT timer interrupt is the canonical mechanism on RISC-V.

How it works.

1. During boot, firmware reads `MTIME` and writes `MTIMECMP[h] = MTIME + tick_interval` for each hart.
2. After `tick_interval` `MTIME` ticks, `MTIP[h]` asserts and the hart traps into `machine_timer_handler`.
3. The handler re-arms the timer:

```
/* from FreeRTOS RISC-V port, simplified */
uint64_t next = mtimecmp[hart_id] + configTICK_RATE_HZ_CYCLES;
if (next < mtime_read()) /* missed deadline */
    next = mtime_read() + configTICK_RATE_HZ_CYCLES;
set_mtimecmp(hart_id, next); /* uses the 0xFFFFFFFF-lo-hi sequence */
```

4. The scheduler runs, picks the highest-priority ready task, and returns to it.

Why the CLINT. MSIP is unused for this pattern; only `MTIME` and per-hart `MTIMECMP` are needed. Each hart in an AMP or SMP configuration gets its own independent tick rate by programming a different `MTIMECMP[h]`.

Real products / projects. FreeRTOS RISC-V port (`port.c`), Zephyr `risc_v_machine_timer` driver, Linux `riscv_timer` clockevent driver.

6.11.2 SMP inter-processor interrupts (IPI)

What it is. In a symmetric multi-processing system, one hart must sometimes force another hart to take an action immediately — reschedule, flush its TLB, execute a remote function call, or yield a lock. The MSIP register is the hardware mechanism for this IPI on RISC-V.

How it works.

Hart A (sender):

1. Enqueue work item in shared memory.
2. Write `MSIP[B] = 1`.

Hart B (receiver, may be in WFI):

1. Running normal code or in WFI.
2. `msip_o[B]` rises → `mip.MSIP` set.
3. Hart B wakes (WFI exits) / traps.
4. `machine_sw_handler()` runs:
 - Reads work queue from shared mem.
 - Processes all pending items.
 - Writes `MSIP[B] = 0` to clear.

Key property. MSIP is a level-sensitive latch — writing 1 a second time before the receiver clears it does **not** queue a second interrupt. The receiver must inspect the shared memory work queue, not assume one IPI equals one work item.

Real products / projects. Linux SMP scheduler (`arch/riscv/kernel/smp.c`), OpenSBI IPI relay (`ipi.c`), Zephyr `ipm_riscv_hart` driver, SiFive Freedom E SDK SMP examples.

6.11.3 Precision elapsed-time measurement

What it is. MTIME is a 64-bit, monotonically-increasing, free-running counter visible from any privilege level (RDTIME instruction maps to it). Firmware uses it as a hardware stopwatch to time code sections, SPI/I2C bit-banging, GPIO pulse widths, or protocol timeouts.

How it works.

```
/* bare-metal benchmark snippet */
uint64_t t0 = mtime_read();           /* snapshot before */
spi_transmit(buf, len);                /* operation under test */
uint64_t t1 = mtime_read();           /* snapshot after */
uint64_t elapsed_ns = (t1 - t0) * MTIME_PERIOD_NS;
```

On a 1 MHz MTIME (1 μ s resolution), a 12 μ s SPI transaction reads as `t1 - t0 = 12`. On the SMC default 10 MHz tick the resolution is 100 ns, sufficient for most driver-level timing.

Key property. MTIME is **shared across all harts**, so timestamps taken on different harts are directly comparable — no per-core timestamp skew as on x86 TSC without synchronisation.

Real products / projects. Embedded motor-control firmware (cycle counting for commutation timing), Linux `riscv_sched_clock()`, GDB stub cycle counters, RISC-V port of Dhrystone / CoreMark benchmarks.

6.11.4 Software watchdog / deadline enforcement

What it is. A software watchdog uses MTIMECMP as a deadline: the main task must “kick” the timer before it expires, or a machine-mode trap handler takes corrective action (log fault, assert reset, switch to a safe state).

How it works.

```
Initialisation:
    MTIMECMP[h] = MTIME + WATCHDOG_TIMEOUT_TICKS    ← arm the watchdog

Main loop (must execute within deadline):
    do_work();
    MTIMECMP[h] = MTIME + WATCHDOG_TIMEOUT_TICKS    ← "kick" (re-arm)

Watchdog trap handler (fires if main loop stalls):
    log_fault(WATCHDOG_TIMEOUT);
    system_reset();
```

Why MTIMECMP. Because MTIP is level-sensitive and re-armed by software, there is no hardware counter to manage — the MTIMECMP write itself is the kick. This makes the watchdog zero-cost in the common (non-fault) path: one 64-bit store per loop iteration.

Real products / projects. Automotive ECU bare-metal firmware, safety- critical RTOS partitions (ISO 26262 / IEC 61508 style), OpenSBI platform fault management.

6.11.5 Timed sleep / WFI with hardware wakeup

What it is. RISC-V defines the `WFI` (Wait For Interrupt) instruction to halt a hart until an interrupt is pending. Combining WFI with `MTIMECMP` produces a low-power, timed sleep with **zero busy-wait overhead**.

How it works.

```
/* put hart to sleep for `ticks` MTIME units */
void sleep_ticks(unsigned hart, uint64_t ticks) {
    set_mtimecmp(hart, mtime_read() + ticks); /* arm wakeup timer */
    enable_machine_timer_interrupt();          /* mie.MTIE = 1 */
    __asm__ volatile("wfi");                  /* hart halts here */
    disable_machine_timer_interrupt();         /* prevent re-entry */
    set_mtimecmp(hart, UINT64_MAX);           /* disarm timer */
}
```

The hart resumes at the WFI instruction (or its successor, depending on microarchitecture) when `MTIP[hart]` asserts. No polling loop, no wasted cycles.

Real products / projects. IoT sensor firmware (nRF52 / GD32VF103 equivalents), Zephyr `k_sleep()` on RISC-V, FreeRTOS tickless idle mode, embedded RTOS power management middleware.

6.11.6 Multi-hart time-base synchronisation

What it is. In heterogeneous or AMP configurations, all harts share the single global MTIME, giving them a **common hardware time reference** without any software clock synchronisation protocol.

How it works. A typical pattern in mixed-criticality systems:

Role	Hart	Action
Safety monitor	Hart 0	Reads MTIME before and after a safety-critical period; asserts fault if $\Delta t > \text{budget}$.
Application core	Hart 1-3	Record MTIME at each phase transition; post timestamps to a shared log.
Debug / trace	Host	Reads MTIME register via <code>transport_dbg</code> or JTAG to correlate software events with hardware waveforms.

Because MTIME is the **only** globally-shared state in the CLINT, and it is read-only from the perspective of normal application software (writes are privileged), there are no coherency concerns — all harts always see the same counter value at the same simulated time.

Real products / projects. SiFive multi-cluster SoCs, BeagleV-Fire (PolarFire SoC), RISC-V hypervisor implementations (virtual MTIME per guest is an extension of this pattern), multi-hart OpenSBI firmware.

6.11.7 Summary — mechanism-to-use-case mapping

CLINT mechanism	Primary use cases
MTIME (read)	Elapsed-time measurement, benchmarking, time-base synchronisation, sleep duration
MTIME (write)	Boot-time initialisation, test harness, debug / calibration
MTIMECMP[h]	Scheduler tick, watchdog deadline, timed sleep wakeup
MSIP[h]	SMP IPI (TLB shutdown, remote reschedule, lock hand-off)

7. Register map

All offsets are relative to the IP's window base. The window is 64 KiB; the SMC platform-level base is `0xC800_0000` per `tt-oca-hw.pdf §6.6.6`.

Offset	Register	Size	SW	Reset value
<code>0x0000 + 4·h</code> , $h \in [0, N)$	MSIP[h]	4 B	RW	<code>0x00000000</code>
<code>0x4000 + 8·h</code> , $h \in [0, N)$	MTIMECMP[h]	8 B	RW	<code>0xFFFFFFFFFFFFFFFF</code> †
<code>0xBFF8</code>	MTIME	8 B	RW	<code>0x0000000000000000</code>

† The Chipyard RTL leaves MTIMECMP without an explicit reset (pad register in `OCAH1CORECluster_CLINT.sv:110`). This model defaults it to all-1s so that `mtip_o[h]` is **deasserted** out of reset until firmware programs a real comparator. See §13 deviation note.

7.1 MSIP[h]

Bits	Field	Type	Description
0	value	RW	IPI flag for hart <code>h</code> . Drives <code>msip_o[h]</code> .
31:1	rsvd0	RAZ	Reserved. Reads return 0; writes are dropped.

Per `clint.rdl`:

```
field { sw=rw; hw=r; } value[0:0] = 0x0;
field { sw=r; hw=r; } rsvd0[31:1] = 0x0;
```

7.2 MTIMECMP[h]

Bits	Field	Type	Description
63:0	count	RW	64-bit comparison value. <code>mtip_o[h] = (MTIME ≥ count)</code> .

Access width: 4 B (low half at `+0`, high half at `+4`) or 8 B at `+0`.

7.3 MTIME

Bits	Field	Type	Description
63:0	count	RW	64-bit free-running counter. Auto-incremented by <code>tick</code> .

Access width: 4 B (low half at `+0`, high half at `+4`) or 8 B at `+0`.

8. Functional behaviour

8.1 MSIP read/write

A 32-bit read of `MSIP[h]` returns `0x0000_0001` if the IPI is set, else `0x0000_0000`. A 32-bit write writes `data[0]` to the IPI bit; bits `[31:1]` are silently dropped.

`msip_o[h]` is updated combinatorially with respect to the register write (one delta-cycle latency in the SystemC model).

8.2 MTIME tick

Every `tick_period_ns` the model:

1. Increments MTIME by 1.
2. Re-evaluates MTIP[h] for every hart.
3. Schedules the next tick.

Setting `tick_period_ns = 0` disables steps 1 and 3; MTIME may then be advanced only via register writes or `dbg_set_mtime`.

8.3 MTIP comparator

For each hart `h`, `mtip_o[h] = (MTIME ≥ MTIMECMP[h])`, evaluated **unsigned**. The output is recomputed whenever:

- MTIME changes (tick or write),
- MTIMECMP[h] changes (write).

The comparator is purely combinational (single delta-cycle settling).

8.4 Single-driver discipline

Both `msip_o[*]` and `mtip_o[*]` are driven exclusively by a single `SC_METHOD (output_method)` sensitive to a `recompute_event_`. Other code paths mutate internal state and post `recompute_event_`; they never touch the output signals. This satisfies the SystemC 3.0 strict single-driver rule and matches the convention used by the PLIC.

9. Reset behaviour

Asserting `rst_n_i` low synchronously resets:

Register / state	Reset value
MTIME	0x0000000000000000
MSIP[h]	0x00000000 for all h
MTIMECMP[h]	0xFFFFFFFFFFFFFFFF for all h
Pending tick event	Cancelled and rescheduled

Outputs `msip_o[*]` and `mtip_o[*]` deassert via the standard `output_method` recompute path — i.e. `reset_proc` posts `recompute_event_` and `output_method` writes the new value.

After `rst_n_i` returns high, the tick event is rescheduled (if `tick_period_ns > 0`) and the model resumes normal operation.

10. Bus interface (TLM-2.0 / AXI4-Lite)

Property	Value
Transport	<code>simple_target_socket<clint></code> (TLM-2.0 LT)
Allowed access widths	4 B (any window register), 8 B (MTIME / MTIMECMP at offset 0)
Alignment	Natural (4 B → <code>addr & 0x3 == 0</code> , 8 B → <code>addr & 0x7 == 0</code>)
Annotated delay	<code>access_delay_ns_p_</code> (default 2 ns; mutable via CCI)
<code>ttransport_dbg</code>	Supported, side-effect-free
DMI	Not granted
Byte enables	Not supported (returns <code>TLM_BYTE_ENABLE_ERROR_RESPONSE</code>)
Streaming width	Must equal <code>data_length</code>

The model accepts both 32-bit and 64-bit accesses because:

- The Chipyard RTL routes 64-bit TileLink accesses to MTIME / MTIMECMP.
- The firmware driver in `riscv_clint0.c` always uses 32-bit accesses (because RV32 hosts cannot issue native 64-bit MMIO).

Both code paths must work against the same model.

11. Error handling

Scenario	Response
<code>addr ≥ WINDOW_SIZE</code>	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
<code>data_length ∉ {4, 8}</code>	<code>TLM_BURST_ERROR_RESPONSE</code>
4-byte access not aligned to 4	<code>TLM_BURST_ERROR_RESPONSE</code>
8-byte access not aligned to 8	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>streaming_width ≠ data_length</code>	<code>TLM_BURST_ERROR_RESPONSE</code>
Byte-enable supplied	<code>TLM_BYTE_ENABLE_ERROR_RESPONSE</code>
8-byte access targeting MSIP	<code>TLM_BURST_ERROR_RESPONSE</code>
Command other than READ / WRITE	<code>TLM_COMMAND_ERROR_RESPONSE</code>
Address inside window but outside any register	RAZ on read, WI on write
Otherwise	<code>TLM_OK_RESPONSE</code>

`transport_dbg` returns 0 (zero bytes transferred) for any of the above error conditions, per TLM-2.0 convention.

12. Programming model

12.1 Reading MTIME (RV32-safe)

```
uint64_t read_mtime(void)
{
    uint32_t hi, lo;
    do {
        hi = mmio_read32(CLINT_BASE + 0xBFFC);
        lo = mmio_read32(CLINT_BASE + 0xBFF8);
    } while (mmio_read32(CLINT_BASE + 0xBFFC) != hi);
    return ((uint64_t)hi << 32) | lo;
}
```

12.2 Setting MTIMECMP[h] (avoid spurious MTIP)

```
void set_mtimecmp(unsigned h, uint64_t value)
{
    /* Park comparator far in the future, then patch low/high. */
    /* Avoids a transient (MTIME >= MTIMECMP) when only the low word has */
    /* been updated. */
    mmio_write32(CLINT_BASE + 0x4004 + 8*h, 0xFFFFFFFFu);
    mmio_write32(CLINT_BASE + 0x4000 + 8*h, (uint32_t)value);
    mmio_write32(CLINT_BASE + 0x4004 + 8*h, (uint32_t)(value >> 32));
}
```


12.3 Sending an IPI

```
void send_ipi(unsigned target_hart) {  
    mmio_write32(CLINT_BASE + 0x0 + 4*target_hart, 1u);  
}  
void clear_ipi(unsigned my_hart) {  
    mmio_write32(CLINT_BASE + 0x0 + 4*my_hart, 0u);  
}
```

12.4 Periodic timer ISR

```
void m_timer_isr(void) {  
    /* read mhartid into h */  
    uint64_t now = read_mtime();  
    set_mtimecmp(h, now + INTERVAL);  
}
```

13. Compliance matrix

Requirement	Source	Status
MSIP only bit[0] writable; bits[31:1] RAZ/WI	<code>clint.rdl</code> §MSIP	OK
MTIMECMP / MTIME regwidth = accesswidth = 64	<code>clint.rdl</code>	OK
MSIP[h] at $0x0 + 4 \cdot h$	<code>clint.rdl</code> addrmap	OK
MTIMECMP[h] at $0x4000 + 8 \cdot h$	<code>clint.rdl</code> addrmap	OK
MTIME at <code>0xBFF8</code>	<code>clint.rdl</code> addrmap	OK
<code>mtip_o[h] = (MTIME ≥ MTIMECMP[h])</code>	<code>OCAH1COREcluster_CLINT.sv:174</code>	OK
<code>msip_o[h] = MSIP[h].bit[0]</code>	<code>OCAH1COREcluster_CLINT.sv:173</code>	OK
MTIME = 0 at reset	RTL <code>time_0 <= 64'h0</code>	OK
MSIP[h] = 0 at reset	RTL <code>ipi_X <= 1'h0</code>	OK
MTIMECMP[h] = <code>0xFFFF...F</code> at reset	Deviation from RTL: the Chipyard RTL leaves <code>pad</code> undefined; we initialise to max so MTIP is deasserted out of reset. Firmware that programs MTIMECMP before unmasking MTIP is unaffected.	Documented deviation
Firmware “high-FF / low / high” MTIMECMP set sequence supported	<code>riscv_clint0.c::__metal_driver_riscv_clint0_mtimecmp_set</code>	OK
Firmware rollover-safe MTIME read sequence supported	<code>riscv_clint0.c::__metal_clint0_mtime_get</code>	OK
32-bit and 64-bit MMIO accesses both supported	Chipyard RTL + firmware	OK
TLM error responses for misaligned / out-of-window / wrong-size	This spec §11	OK
DMI refused	This spec §10	OK
CCI 1.0 parameterisation (<code>num_harts</code> , <code>tick_period_ns</code> , <code>access_delay_ns</code>)	This spec §4	OK
Per-hart isolation	This spec §6.5	OK

14. Revision history

Date	Version	Author	Notes
2026-05-11	0.1	SMC modelling	Initial release.

15. Glossary

Term	Definition
CLINT	Core-Local Interruptor — the per-hart timer + software-interrupt controller.
CSR	Control/Status Register — RISC-V's privileged register space.
CCI	Configuration, Control & Inspection — Accellera's SystemC parameterisation API.
Hart	Hardware thread (a single RISC-V execution context).
IPI	Inter-Processor Interrupt — one hart signalling another via MSIP.
MSIP	Machine Software Interrupt Pending; per-hart 1-bit flag in CLINT.
MTIME	64-bit free-running counter; the global time base.
MTIMECMP	Per-hart 64-bit comparator. $mtip_o[h] = (MTIME \geq MTIMECMP[h])$.
MTIP	Machine Timer Interrupt Pending; the line driven by the comparator.
PLIC	Platform-Level Interrupt Controller (sister IP; handles every other IRQ).
RAZ/WI	Read-As-Zero / Write-Ignored.
RDL	Register Description Language (SystemRDL); see <code>clint.rdl</code> .
TLM	Transaction-Level Modeling — SystemC's untimed / loosely-timed bus abstraction.